

CIMA0 模块演化与实现规则

模块归属优先原则

系统出现任何现象、问题或不足时，不直接增加规则、控制逻辑或新模块。

必须遵循：

发现现象

↓

分析现象属于哪一种能力

↓

定位负责模块

↓

修改该模块内部表达结构

↓

重新进行设计原则审计

内部表达结构原则

模块优化不是通过外部干预改变行为，而是通过调整模块自身的表达能力，使系统自然产生新的行为。

禁止：

- 通过外部控制修正结果；
- 增加针对特定现象的特殊规则；
- 使用中心模块协调局部模块；
- 为了达到预期结果添加人为目标。

允许：

- 修改状态表达；
- 修改局部变量结构；
- 修改内部关系表达；
- 修改局部交互形式；
- 修改时间尺度表达。

目标不是控制结果，而是让结构能够承载更多自然演化。

四模块问题归属规则

内部动力模块

负责：

- 状态表达；
- 局部相互作用；
- 动力演化；
- 时间推进。

当问题涉及：

- 演化轨迹；
- 稳定性；
- 局部耦合；
- 自发变化；

只修改动力模块内部表达。

IO模块

负责：

- 外界扰动进入；
- 原始信息边界传递；
- 输入输出边界保持。

当问题涉及：

- 输入格式；
- 字节流处理；
- 边界转换；

只修改IO内部表达。

禁止：

- 语义解释；
 - 目标识别；
 - 智能判断。
-

采样模块

负责：

- 有限观察；
- 局部投影；

- 稀疏扫描；
- 精度层级。

当问题涉及：

- 如何观察；
- 如何采样；
- 如何形成瞬时快照；

只修改采样结构。

禁止：

- 控制动力；
- 决定系统目标；
- 拥有全局视角。

计算模块

负责：

- 自身计算资源；
- 计算预算；
- 精度投入；
- 举手机制。

当问题涉及：

- 是否增加计算；
- 是否提高采样精度；
- 当前资源分配；

只修改计算模块内部表达。

禁止：

- 判断世界重要性；
- 修改动力状态；
- 成为中央控制器。

禁止跨模块修补原则

发现问题时：

禁止：

动力问题 → 修改Observer

采样问题 → 修改Planet

计算问题 → 修改IO

输入问题 → 增加Cloud控制层

正确：

问题属于谁

↓

谁内部演化

↓

谁负责表达

最终审计规则

任何代码修改完成后，必须检查：

1. 是否引入全局状态？
2. 是否产生隐藏控制者？
3. 是否模块越权？
4. 是否增加非必要结构？
5. 是否通过改变表达结构解决问题，而不是通过外部修正？

只有全部通过，才进入下一阶段。

核心思想：

系统不是通过外部管理获得复杂性。

复杂性来自：

最小局部结构

↓

内部表达变化

↓

局部相互作用

↓

大尺度涌现